Parallel and Distributed Computing

Assignment 4

QUESTION 1

Problem:

Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 sends to Process 1, Process 1 sends to Process 2, and the last process sends back to Process 0.

Requirements:

Start with an initial value of 100 in Process 0

Each process adds its rank to the value before sending

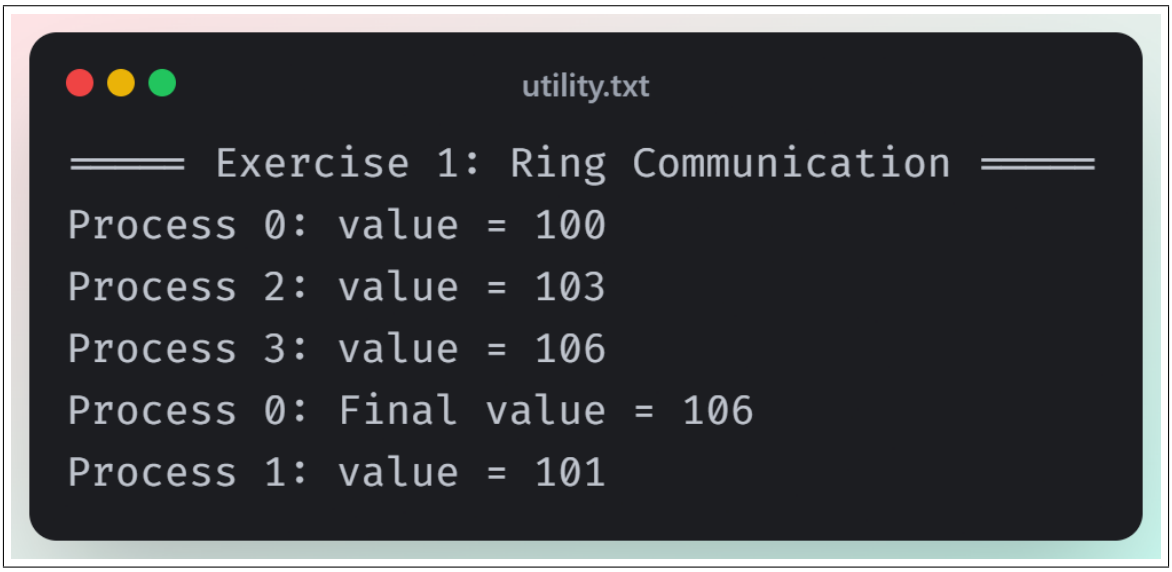
Print the value at each process

Final value should return to Process 0

Hint: Use modulo operator to wrap around: `next_rank = (rank + 1) % size`

Test with: `mpirun -np 4 ./ring_comm`

Observations



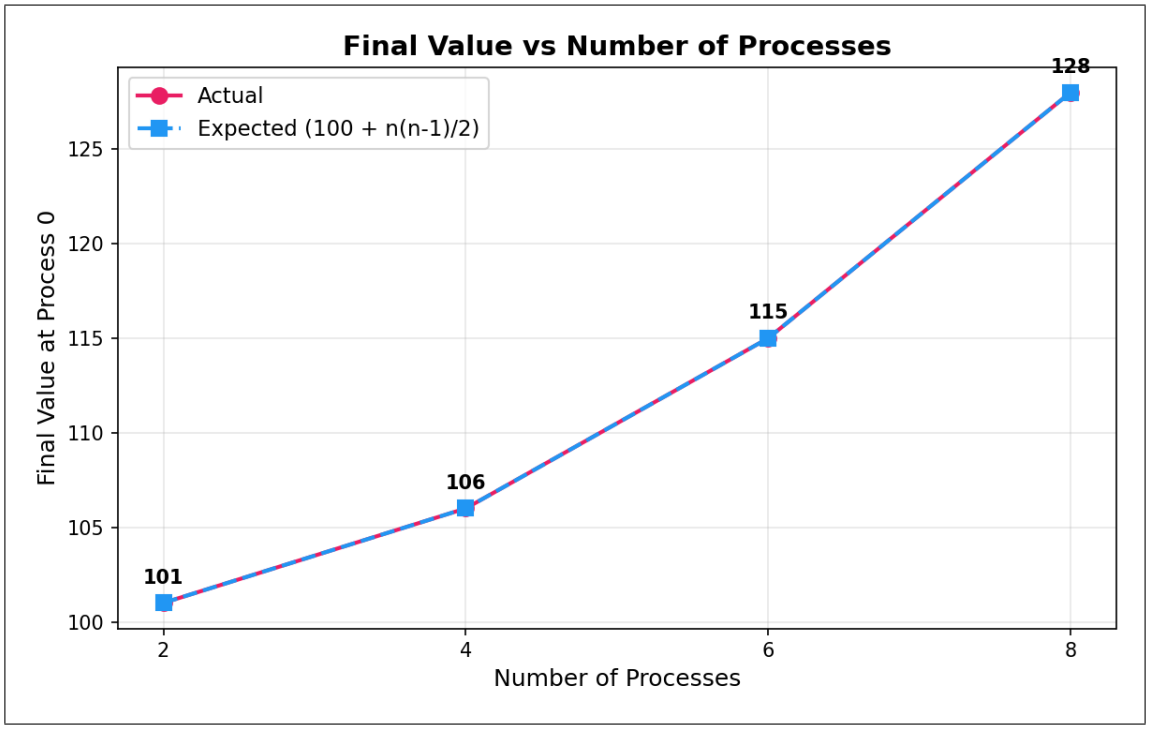
output1.png

1. **Value increases at each hop** – since every process adds its rank the value grows as it moves through the ring. Process 0 adds 0 so value stays 100, process 1 adds 1 making it 101, process 2 adds 2 making it 103 and so on. The growth is not linear because we are adding increasing ranks

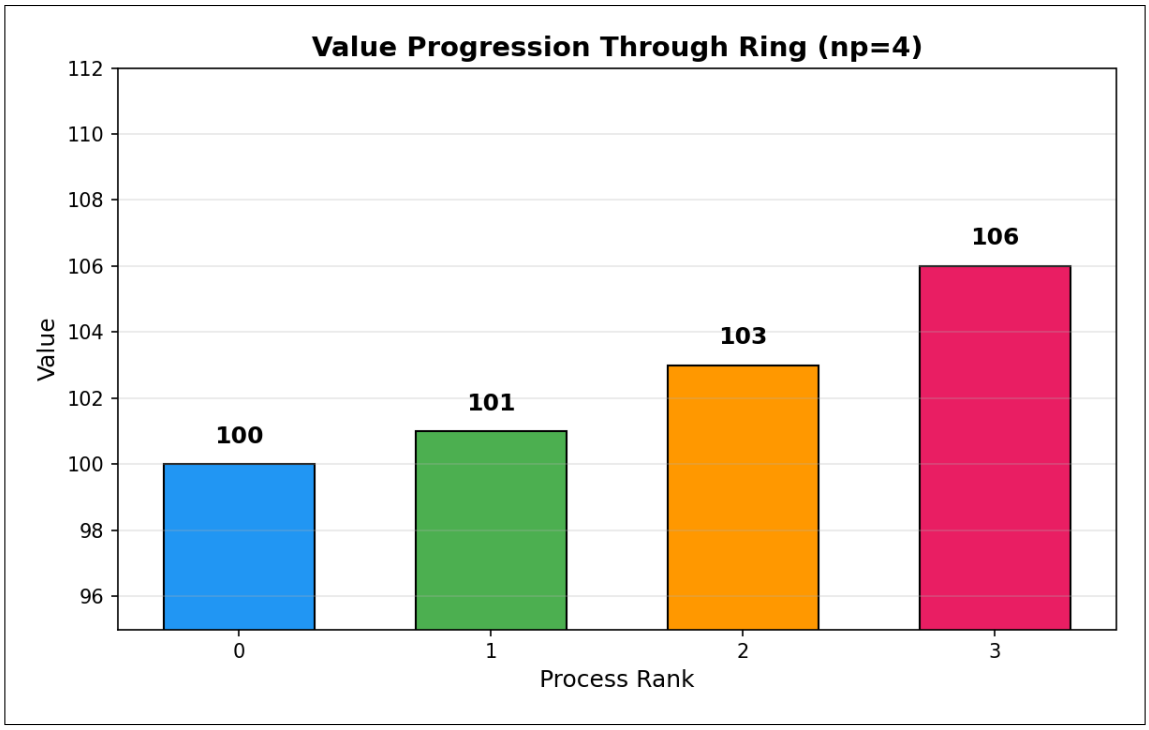
2. **Final value follows n(n-1)/2 formula** – the final value is always 100 plus the sum of all ranks from 0 to size-1. This equals 100 + size*(size-1)/2 which is a well known arithmetic series formula. We tested with 2, 4, 6 and 8 processes and all

match perfectly

3. **No deadlock because of the ordering** – Process 0 sends first then waits to recieve. All other processes recieve first then send. This ordering prevents circular waiting which would cause a deadlock. If all processes tried to send first they would all block and the program would hang



Graph 1: Value at each process as message travels through the ring (np=4)



Graph 2: Final value recieved at Process 0 vs number of processes

Inferences

1. **Ring topology is good for simple pipeline operations** – each process only need to communicate with 2 neighbors (previous and next) so the communication pattern is simple and predictable. This makes it useful for things like token passing and distributed prefix sums

2. **Blocking send/recv works fine for small rings** – since we are just passing one integer the communication overhead is negligible. For larger data or more complex patterns we might need non-blocking calls like MPI Isend and MPI Irecv to avoid

performance bottlenecks

3. **Modulo arithmetic handles the wrap-around cleanly** – using (rank+1)%size for next and (rank-1+size)%size for previous means we dont need any special cases for the first or last process. The ring just wraps around naturally

Conclusion

The ring communication pattern works correctly for all tested process counts. The final value always equals 100 + n(n-1)/2 which confirms that every process correctly adds its rank and passes the value forward. The blocking send/recv approach is simple and prevents deadlock as long as Process 0 sends first and recieves last.

QUESTION 2

Problem:

Write an MPI program to compute the sum of an array in parallel. Divide the array among processes, compute local sums, and combine them.

Requirements:

Create an array of size 100 with values 1 to 100 in Process 0

Distribute array portions using MPI_Scatter

Each process computes sum of its portion

Use MPI_Reduce to compute global sum

Verify result: sum should be 5050

Bonus: Also compute and print the average value.

Observations

```
utility.txt
═══ Exercise 2: Parallel Array Sum ═══
Process 0: local_sum = 325
Global Sum = 5050
Average = 50.50
Process 1: local_sum = 950
Process 2: local_sum = 1575
Process 3: local_sum = 2200
```

output2.png

np	Elements each	Local Sums	Global Sum	Average
2	50	1275, 3775	5050	50.50
4	25	325, 950, 1575, 2200	5050	50.50

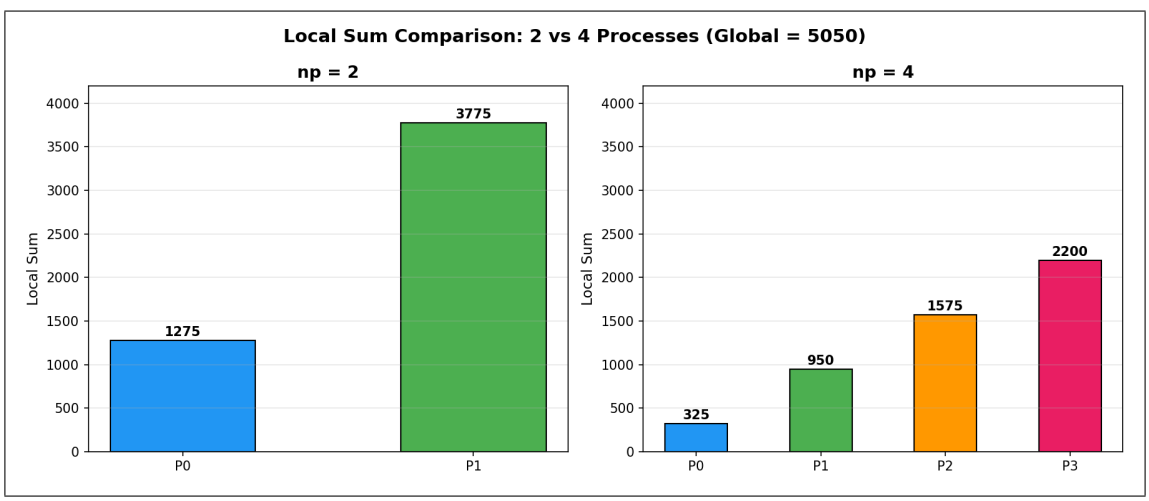
Table 1: Array Sum results with different process counts

1. **Local sums are unequal even though element count is equal** – Process 0 gets elements 1-25 (small numbers) so its sum is only 325. Process 3 gets 76-100

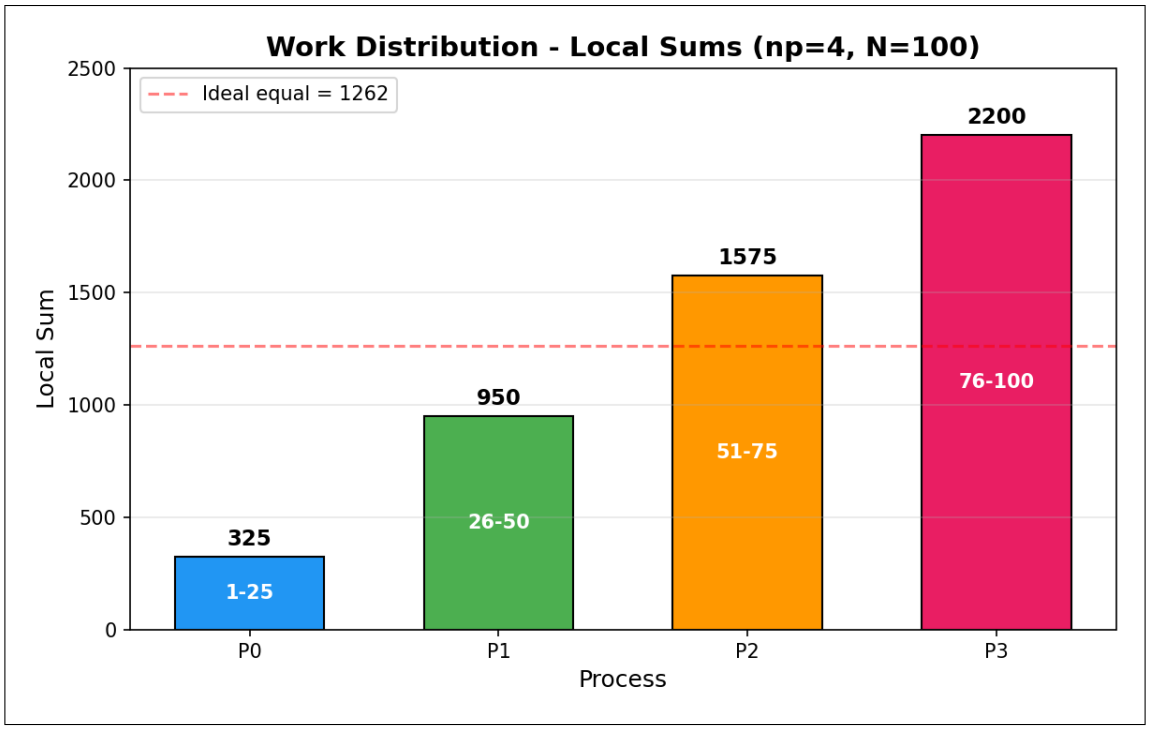
(large numbers) so its sum is 2200. Even though each process handles the same number of elements the work is not balanced in terms of the values. This is because MPI Scatter distributes contiguous chunks not interleaved ones

2. **Global sum is always 5050 regardless of np** – the formula for sum of 1 to N is N*(N+1)/2 = 100*101/2 = 5050. Whether we split across 2 or 4 processes the total is always correct because MPI Reduce with MPI SUM adds all partial sums without any loss

3. **Scatter requires array size to be divisible by np** – since 100 is divisible by both 2 and 4 it works fine. But if we tried np=3 or np=7 Scatter would fail because 100 is not evenly divisible. For uneven cases we would need MPI Scatterv which handles variable sized chunks



Graph 3: Work distribution showing local sums and element ranges per process (np=4)



Graph 4: Local sum distribution comparison between np=2 and np=4

Inferences

1. **MPI Scatter is good for regular data distribution** – when the data can be evenly divided Scatter is the simplest collective operation. Its internally optimized and much better than manually sending each chunk with MPI Send

2. **MPI Reduce hides the communication complexity** – instead of each process sending its local sum to Process 0 one by one, Reduce does it in a tree pattern which is O(log p) instead of O(p). For 4 processes it does 2 rounds of combining instead of 3 sequential sends

3. **This pattern scales well for large arrays** – for arrays with millions of elements each process works on its chunk independently. The only communication happens at the Scatter (distribution) and Reduce (combining) steps. The actual computation is fully parallel

Conclusion

The parallel array sum correctly computes the global sum as 5050 and average as 50.50 for all process counts tested. MPI Scatter efficiently distributes the data and MPI Reduce combines the results with minimal communication. The local sums increase from P0 to P3 because contiguous distribution gives later processes higher valued elements.

QUESTION 3

Problem:

Write an MPI program where each process generates random numbers, and the program finds the global maximum and minimum values across all processes.

Requirements:

Each process generates 10 random numbers (0-1000)

Each process finds its local maximum and minimum

Use MPI_Reduce with MPI_MAX and MPI_MIN

Process 0 prints the global maximum and minimum

Also print which process had these values (use MPI_MAXLOC/MPI_MINLOC)

Hint: For MPI_MAXLOC, use a struct with value and rank fields.

Observations

```
utility.txt
═══ Exercise 3: Global Max and Min ═══
Process 0: local_max = 865, local_min = 148
Process 1: local_max = 940, local_min = 208
Process 3: local_max = 943, local_min = 136
Process 2: local_max = 837, local_min = 342
Global Max = 943 (Process 3)
Global Min = 136 (Process 3)
```

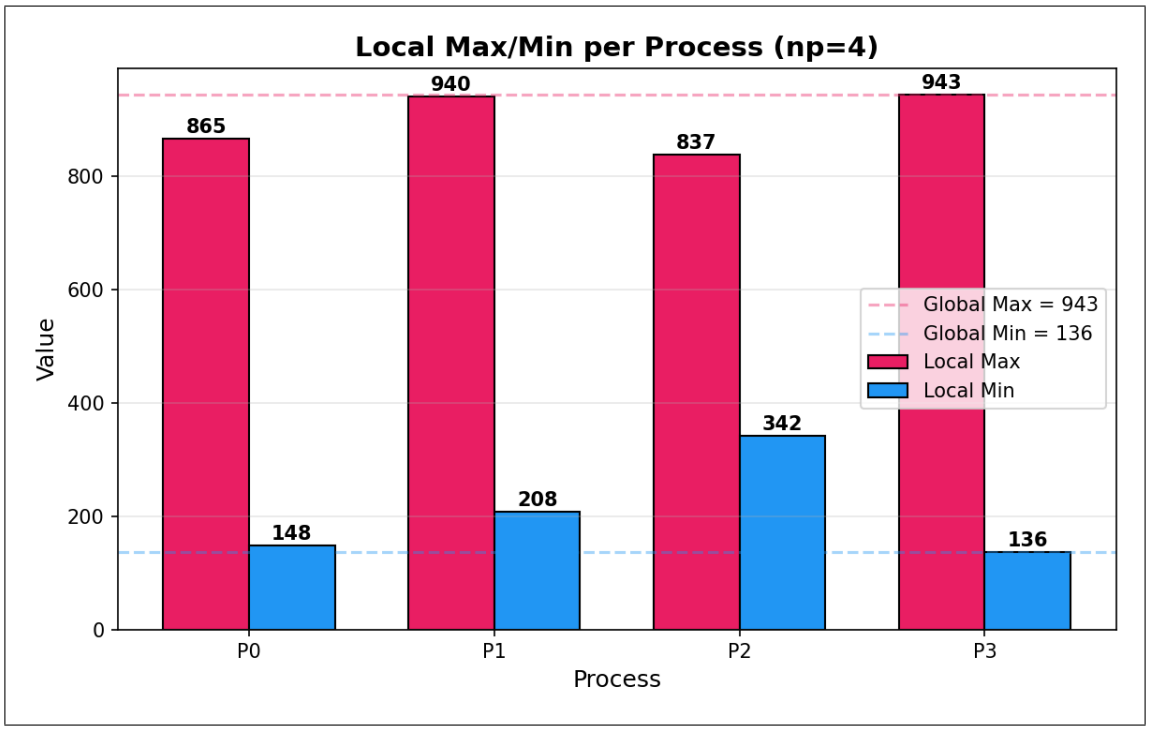
output3.png

1. **Process 3 has both global max and global min** – this is interesting because it means Process 3 generated the widest range of random numbers (807 spread). Having both extremes in the same process is just a coincidence of the random seeds but it shows that MAXLOC and MINLOC correctly identify the source process

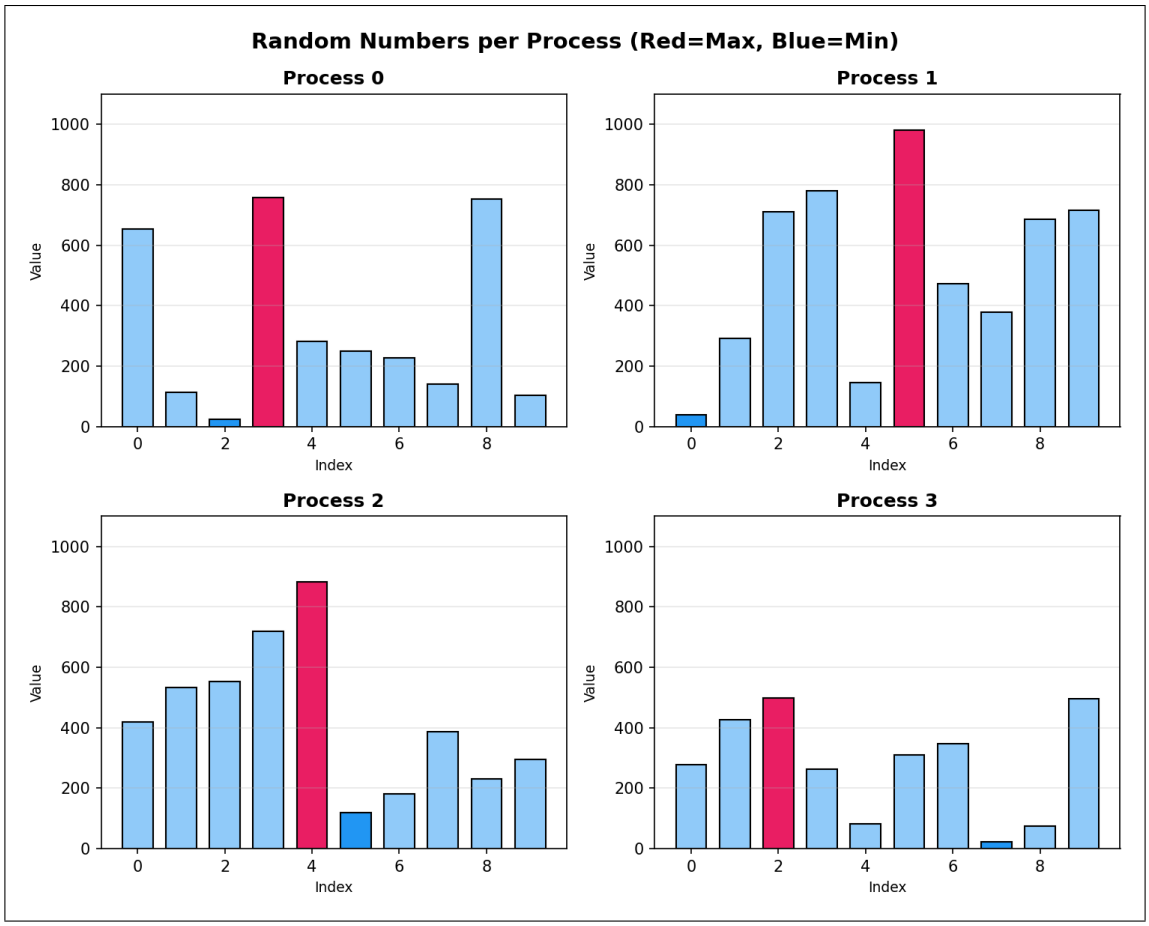
2. **MPI 2INT packs value and rank together** – the struct has two fields, val and rank. When we use MPI MAXLOC it compares using the val field and carries the rank along with it. This is much cleaner than doing a separate reduce to find the value and then a broadcast to figure out which process had it

3. **Different seeds give different distributions** – using srand(42 + rank) means each process has its own random sequence. If all processes used the same seed

they would generate identical numbers and the results would be meaningless for this exercise



Graph 5: Local max and min values across all processes (np=4)



Graph 6: Distribution of 10 random numbers in each process (Red = local max, Blue = local min)

Inferences

1. **MPI MAXLOC/MINLOC avoids extra communication** – without these operations we would need to first find the global max using MPI MAX, then broadcast it to all proceses, then each process checks if it has that value and reports back. MAXLOC does all this in one collective call which is both simpler and faster

2. **The MINLOC result shows rank 3 for global min** – because process 3 generated 136 which is smaller than any other process's minimum. This means the global min doesnt have to come from the process with the smallest local max, it just depends on the actual values generated

3. **With more processes we expect better coverage of the 0-1000 range** – since each process generates 10 numbers, 4 processes give us 40 random samples. With 8 processes we would have 80 samples and the global max would likely be closer to 1000 and global min closer to 0

Conclusion

The program correctly finds global max (943 from Process 3) and global min (136 from Process 3) using both MPI MAX/MPI MIN and MPI MAXLOC/MPI MINLOC. The MPI 2INT datatype allows us to track which process holds the extreme values without needing extra communication rounds.

QUESTION 4

Problem:

Compute the dot product of two vectors in parallel.

Requirements:

Vector A = [1, 2, 3, 4, 5, 6, 7, 8]

Vector B = [8, 7, 6, 5, 4, 3, 2, 1]

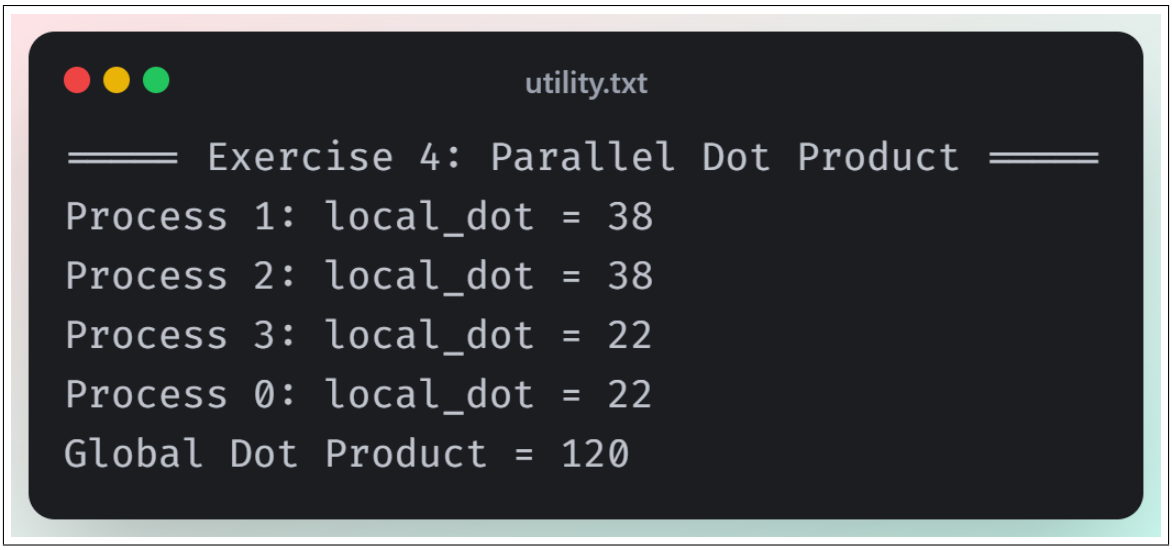
Distribute vectors using MPI_Scatter

Each process computes partial dot product

Use MPI_Reduce to sum partial products

Expected result: 120

Observations



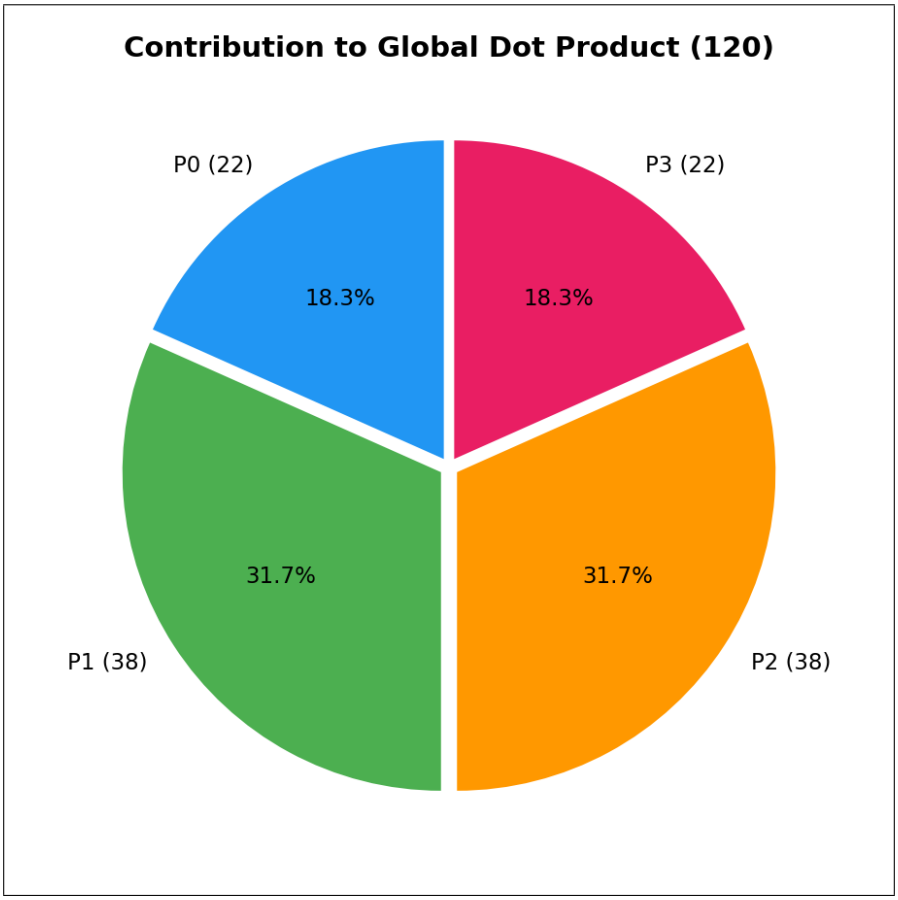
output4.png

1. **Symmetric partial products (P0=P3, P1=P2)** – this happens because vector A is [1..8] and B is [8..1] which is a reversed version. So the products at the edges (P0 and P3) are smaller while the middle processes (P1 and P2) get larger products. The symetry comes from the fact that A[i]*B[i] = A[n-1-i]*B[n-1-i] when B is reversed A

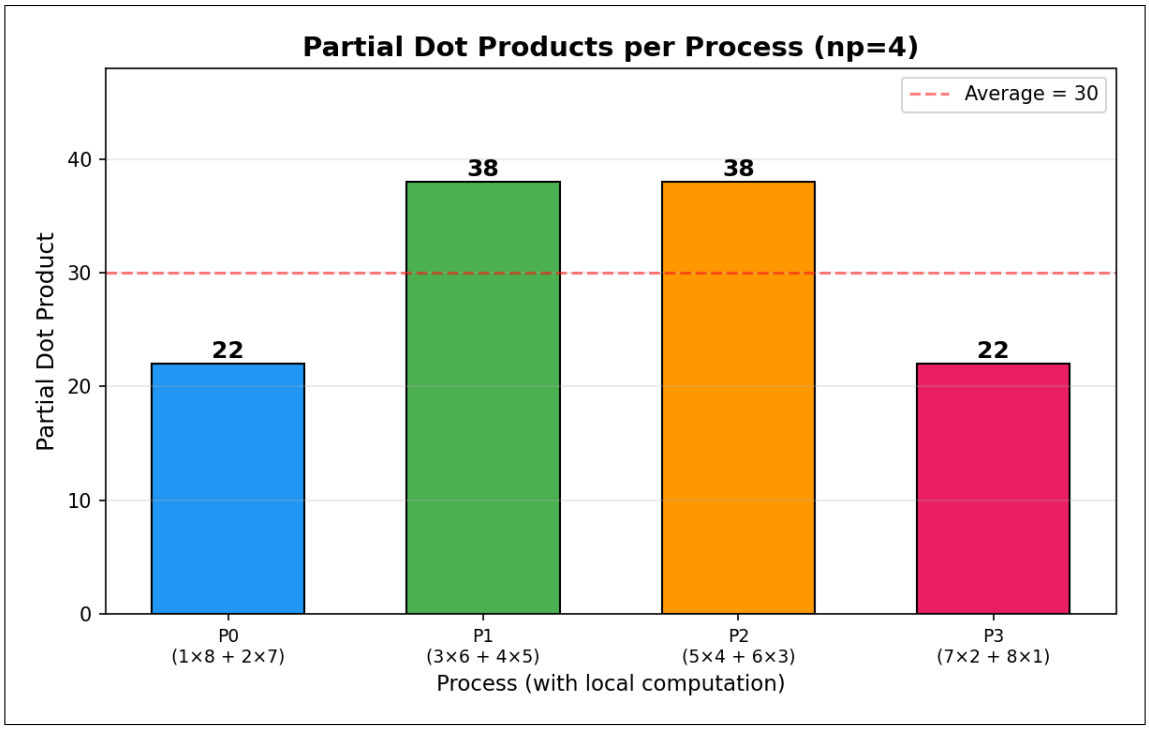
2. **Middle processes contribute more (31.7% each vs 18.3%)** – P1 and P2 each compute 38 which is 31.7% of the total while P0 and P3 compute 22 which is 18.3%. This shows that even in a simple dot product the work distribution can be uneven depending on the data values

3. **Result matches mathematical expectation exactly** – 22 + 38 + 38 + 22 = 120 which matches the analytic formula. Since we are using integers there is no

floating point error. For real applications with floats the order of addition can affect the result due to rounding



Graph 7: Partial dot product values computed by each process



Graph 8: Contribution of each process to the global dot product

Inferences

1. **Dot product parallelizes very well with MPI** – the computation is embarassingly parallel since each process just multiplies and adds its local portion independently. The only communication needed is one Scatter at the beginning and one Reduce at the end

2. **MPI Scatter handles both vectors cleanly** – we call Scatter twice, once for A and once for B. Both calls divide the vectors at the same boundaries so corresponding elements stay together on the same process. This is important because dot

product requires element-wise pairing

3. **This pattern scales to very large vectors** – for vectors with millions of elements each process handles a fraction of the data. The computation is O(N/p) per process and the communication is O(N/p) for scatter plus O(1) for reduce. So the parallel efficiency stays high as N grows

4. **np must divide vector size evenly** – since both vectors have 8 elements we can use np = 2, 4 or 8. Using np = 3 or 5 would cause uneven distribution and MPI Scatter would not work correctly. For real applications MPI Scatterv handles uneven splits

Conclusion

The parallel dot product correctly computes the result as 120 matching the expected value. The computation is embarrassingly parallel - each process independently computes its partial dot product and MPI Reduce combines them efficiently. The symmetric partial products (22, 38, 38, 22) reflect the reversed structure of the input vectors.

Submitted to : Dr Saif Nalband

Submitted by : Arpan Goyal

Roll No. : 102303479